GraphRAG for Organizational Knowledge

Veritask Sciences — Project Documentation

The Gen Academy | *Mastering Agentic AI Bootcamp*

Week 2 Project — Build Your RAG Application

Track 2: Code-heavy (LangChain + LangGraph) | Use Case 3: GraphRAG for Org Knowledge

Prepared by: Prachet Mohanty

Repository: github.com/prachetmohanty/GenAcademy-GraphRAG-for-Organizational-Knowledge

Table of Contents

1. Workshop Brief — What Was Asked

This project was completed as part of Week 2 of The Gen Academy's Mastering Agentic AI Bootcamp. The workshop handout (“Build Your RAG Application”) asked each participant to make two independent choices and then deliver a working, evaluated RAG system.

1.1 The Two Choices

Choice	Option Selected	Description
Use case	Suggested Use Case #3: GraphRAG for Org Knowledge	Model an organization's people, projects, skills, tools, documents, and decisions as a graph, and answer relationship-style queries that plain vector search struggles with.
Build track	Track 2: Code-heavy with LangChain + LangGraph	Python primitives (loaders, embeddings, retrievers) plus LangGraph for a stateful, multi-step GraphRAG flow.

1.2 Required Deliverables (per the Week 2 handout)

- A knowledge graph with 20+ nodes covering people, projects, skills, tools, documents, and decisions.
- A comparison of GraphRAG vs. traditional vector-based RAG on the same 10 queries, with recall against gold-standard entity sets.
- An analysis of when each approach wins (i.e., when relationships matter more than semantic similarity, and vice versa).
- Project documentation: overview, datasets used, prompts used during “vibe coding,” iterations tried, and learnings.
- A video demo (≤ 5 minutes) walking through the application and the build process.
- A GitHub repository containing the code, data, and setup files.
- At least one model call routed through a designated inference provider (originally specified as Nebius Token Factory; this build uses Groq as a substitute — see Section 5.4).

1.3 The RAG Framework (as required by the handout)

Before any build work, the handout requires a one-line scope statement and a six-field framework covering use case, corpus, ingestion/cleaning, ingestion/freshness, chunking/embedding, and retrieval strategy. Section 3 of this document fills in that framework for our chosen company and use case.

1.4 The Reference Solution Kit

The Gen Academy also provided an optional “hint” solution kit (code + demo video) for this use case, built around a generic mock organization. Per the handout's instructions, this kit was used only as an architectural reference — to understand the expected pipeline shape (vector RAG baseline vs. GraphRAG, entity linking, subgraph expansion, Streamlit comparison UI) — not replicated. Our build uses an original fictional company, an original mock dataset, an original set of 10 benchmark questions, and a from-scratch implementation adapted to a pharma/regulatory domain.

2. Project Overview

2.1 The Company: Veritask Sciences (Fictional)

Veritask Sciences is a fictional digital-product and AI-implementation consultancy that builds safety, pharmacovigilance, and regulatory-compliance data products for pharmaceutical and biotech clients. Typical engagements include adverse-event signal-detection platforms, SOC2 / 21 CFR Part 11 audit readiness, regulatory submission tooling, and pricing/operations work for the consultancy itself.

This domain was chosen because it naturally produces the kind of cross-team, “who/what/why” organizational questions that a knowledge graph is well suited to answer — e.g., who worked on a compliance audit, which tools a project used, who approved a pricing decision, and which documents reference a given decision.

2.2 The One-Liner

My RAG app helps **Veritask program managers, tech leads, and executives** answer **cross-team “who / what / why” questions about projects, decisions, tools, and approvals** from **Veritask's internal knowledge base (mock project tracker, MS Teams exports, wiki pages, compliance documents, and a people directory — 5 sources, 33 entities)** in **a Streamlit comparison app** with **100% recall on relationship-style questions versus a vector-RAG baseline**.

2.3 Summary of What Was Built

- A mock organizational knowledge graph for Veritask Sciences with 33 nodes (people, projects, skills, tools, documents, decisions) and 63 relationships.
- Two parallel answer pipelines over the same data: a Vector RAG baseline (FAISS + local embeddings) and a GraphRAG pipeline (NetworkX graph + LangGraph state machine).
- A deterministic entity-linking safety net (regex + topic-shortcut map) that achieves 100% recall on its own, plus an LLM-based linker as the primary path.
- 10 original benchmark questions with hand-curated gold entity sets, split between multi-hop relationship questions (where GraphRAG should win) and single-entity attribute questions (where both pipelines are competitive).
- A command-line comparison tool (compare.py) that runs both pipelines over all 10 questions and reports recall.
- A Streamlit app (ui.py) that runs both pipelines side by side for a chosen question, visualizes the knowledge graph, and highlights which entities each pipeline retrieved.

3. The RAG Framework (Filled)

The handout requires every field below to be filled in 1–2 sentences, forcing an explicit decision at each layer of the RAG stack. Our completed framework for Veritask Sciences:

Field	Our Answer
Use case	PMs and tech leads at Veritask ask multi-hop questions (“Who worked on the SOC2 audit and what tools did they use?”, “Who approved the pricing model change?”) in a Streamlit demo app that compares GraphRAG vs. vector RAG side by side.
Corpus	Five mock sources, ~33 entities total: (1) a project tracker (projects + status + owners), (2) an MS Teams export (decision/approval threads), (3) internal wiki pages (tool and skill descriptions), (4) compliance/audit documents (SOC2 report, pricing memo, security policy), and (5) a people directory (roles, skills). All English, all owned by “Veritask Ops”, all defined in one Python module for reproducibility.
Ingestion + cleaning	Each source is modeled as a list of structured dicts (id, type, name, attributes, text) rather than raw files, so the “cleaning” step is design-time: every node already has a clean, boilerplate-free text field used for both embedding and graph-fact serialization.
Ingestion + freshness	For the demo, graph_builder.py performs a one-time batch build at startup. In a production deployment this would be a nightly refresh from the project tracker API and Teams export, with a 24-hour freshness SLA.
Chunking + embedding	Each graph node is flattened into one short text chunk (type + name + description), ~100–300 tokens — no further sub-chunking is needed because nodes are already atomic units of meaning. Embeddings are produced locally with the sentence-transformers all-MiniLM-L6-v2 model (384-dim), chosen because the chunk set is small (33 chunks) and uniform in size, so a lightweight model gives good separation at zero API cost.
Retrieve	Dual pipeline: (a) Vector RAG baseline — FAISS flat index, dense cosine similarity, top-8; (b) GraphRAG — NetworkX MultiDiGraph, entity-linking (LLM + deterministic fallback) seeds a 2-hop subgraph expansion, optionally filtered to specific relation types (WORKED_ON, USED_TOOL, APPROVED, DECISION_FOR, etc.), with document text attached for prose detail.

3.1 The “I Don't Know” Path

The handout stresses that the refusal path matters more than the happy path. In this build:

- GraphRAG: if entity linking finds zero seed nodes — both the LLM linker and the deterministic fallback — the pipeline responds “I could not find this in our knowledge graph—no relevant entities were linked to this question” instead of guessing.
- Vector RAG: the generation prompt explicitly instructs the model to say so if the retrieved chunks don't contain enough information, rather than fabricating an answer.

4. User Personas

The use case (“GraphRAG for Org Knowledge”) is explicitly framed for program/project managers, executives, consultants, and tech leads. To make the design concrete, we defined four personas at Veritask Sciences. Each persona maps to a real subset of our 33-node knowledge graph and to specific questions in our 10-question benchmark set (see Section 7).

4.1 Priya Subramaniam — Product Manager

Role	Product Manager
Summary	Priya owns the roadmap for Veritask's regulatory and data-quality products. She frequently needs to answer “who is doing what, and what did we decide” questions when planning sprints, writing status updates, or onboarding new team members — often spanning multiple projects she doesn't work on day-to-day.

Goals

- Quickly find who owns or has worked on a given project, without digging through Teams threads.
- Understand the status and ownership of cross-functional initiatives (pricing, regulatory submissions) at a glance.
- Trace a decision back to who approved it and which document records it, for stakeholder updates.

Pain Points with Traditional Vector Search

- Vector search on “who approved the pricing decision” often returns the pricing memo itself, but not the approver's name if it's stored as a separate person record with little lexical overlap.
- Project ownership and status are attributes scattered across the tracker and Teams — a single semantic search rarely connects “project” to “owner” to “decision” in one hop.

Representative Questions in Our Benchmark Set

- “What is the status of the Regulatory Submission Assistant project and who owns it?” (Q9)
- “What decisions were made about the pricing model and who approved them?” (Q2)

Why GraphRAG Helps This Persona

- GraphRAG traverses OWNS and DECISION_FOR / APPROVED edges directly, so Priya gets the project, its owner, the decision, and the approver in one answer — not just the most semantically similar document.

4.2 Grace Adeyemi — VP of Engineering (Executive)

Role	VP of Engineering (Executive)
Summary	Grace approves major technical and security decisions and needs a fast way to audit “what did I (or my org) decide, and why” — especially ahead of board updates, client audits, or when a new hire asks why a particular tool or policy is in place.

Goals

- Review the full list of decisions she has personally approved, across security, ML tooling, and audits.
- Connect a decision to the people, projects, and documents that justify it — useful for compliance traceability.
- Get a defensible, source-grounded answer rather than a plausible-sounding guess when asked by auditors.

Pain Points with Traditional Vector Search

- A vector search for “VP Engineering decisions” may retrieve documents that mention her role, but won't reliably enumerate every APPROVED edge in the graph — recall on “all decisions by person X” is structurally weak for embeddings.
- Executives often ask compound questions (“who is X and what did they approve”) that mix an attribute lookup with a relationship traversal — a single embedding search optimizes for neither.

Representative Questions in Our Benchmark Set

- “Who is the VP Engineering and which decisions did they approve?” (Q8)
- “Who approved the decision to mandate MFA for production systems?” (Q5)

Why GraphRAG Helps This Persona

- GraphRAG's entity linker resolves “VP Engineering” to Grace Adeyemi as a node, then traverses every APPROVED edge from that node — returning a complete, auditable list rather than whatever happened to be semantically closest.

4.3 Arjun Mehta — Senior ML Engineer / Tech Lead

Role	Senior ML Engineer / Tech Lead
Summary	Arjun leads the ML work behind the Signal Detection Platform. He needs to quickly recall technical decision history (“why did we standardize on PyTorch?”), find related architecture documents, and identify who else has the skills or context to help on a given project.

Goals

- Find the architecture spec and authors for a project he's ramping up on.
- Trace a tooling decision back to who proposed it and which project it was first applied to.
- Identify colleagues with overlapping skills (e.g., ML Engineering, Data Engineering) for staffing or pairing.

Pain Points with Traditional Vector Search

- “Which project is the architecture spec for, and who authored it?” requires hopping from a Document node to a Project node and then to Person nodes — three different entity types that a flat vector index treats as independent chunks.
- Tooling decisions (“standardize on PyTorch”) and their proposers are often recorded in different places (a decision log vs. a person's project history); semantic similarity alone won't reliably connect “PyTorch” to “Arjun Mehta” unless his bio happens to mention PyTorch by name.

Representative Questions in Our Benchmark Set

- “Which project is the Signal Detection Platform's architecture spec related to, and who authored it?” (Q3)
- “What tools were standardized for ML training pipelines and who proposed that decision?” (Q6)

Why GraphRAG Helps This Persona

- GraphRAG's REFERENCES, AUTHORED, USED_TOOL, and PROPOSED edges let Arjun jump from a document to its project to its authors to the tools and decisions associated with that project — the exact multi-hop chain a tech lead actually thinks in.

4.4 Devon Park & Nina Castellanos — Security Engineer & Compliance Lead (Cross-Team ICs)

Role	Security Engineer & Compliance Lead (Cross-Team ICs)
Summary	Devon (Security Engineer) and Nina (Compliance Lead) jointly own audit-readiness work. They need to answer “who touched this audit, what evidence exists, and what tools/policies back it up” — exactly the kind of question that shows up during a real SOC2 / Part 11 audit.

Goals

- Enumerate everyone who worked on the SOC2 audit and the tools they used as evidence sources.
- Connect security policies (e.g., MFA mandate) to the documents and decisions that codify them.
- Produce a defensible “chain of custody” narrative for auditors: person → project → tool / decision → document.

Pain Points with Traditional Vector Search

- Audit questions are inherently relational (“who + what tools + which decision + which document”) — a vector search optimized for topical similarity tends to surface the audit report itself repeatedly while missing the people and decision nodes.
- Security/compliance terminology (“MFA”, “SOC2”, “Part 11”) is often abbreviated inconsistently across sources, which hurts embedding-based matching but is easy to encode as a deterministic topic shortcut in a graph linker.

Representative Questions in Our Benchmark Set

- “Who worked on the SOC2 audit and what tools did they use?” (Q1)
- “Which security tools are used across projects and who is the security engineer?” (Q10)

Why GraphRAG Helps This Persona

- The deterministic fallback linker maps topic terms like “audit”, “soc2”, and “mfa” directly to the relevant project, document, and decision nodes, then 2-hop expansion pulls in every WORKED_ON, USED_TOOL, AUTHORED, and APPROVED edge — giving a complete audit trail in one query.

5. What We Built, and Why

5.1 High-Level Architecture

Both pipelines answer the same question over the same mock dataset, so the comparison isolates retrieval strategy rather than data differences:

Vector RAG (Baseline)	GraphRAG (LangGraph)
1. Flatten each of the 33 graph nodes into one text chunk.	1. link_entities — LLM extracts entity names + relation types from the question.
2. Embed all chunks locally (sentence-transformers, all-MiniLM-L6-v2) and store in a FAISS flat index.	2. fallback_linker — if no seeds found, deterministic regex + topic-shortcut map finds seed nodes.
3. Embed the question and retrieve the top-8 most similar chunks (cosine similarity).	3. expand_subgraph — 2-hop breadth-first traversal from seeds, optionally filtered to specific relation types; retries without the filter if it returns zero edges.
4. Prompt the LLM (Groq) to answer using only the retrieved chunks, naming every relevant entity, and to say “not found” if insufficient.	4. attach_documents — pull full text of any Document nodes inside the subgraph for prose detail.
	5. generate_answer — prompt the LLM (Groq) treating graph facts as primary truth and documents as supporting detail.

5.2 The LangGraph State Machine (GraphRAG)

The GraphRAG pipeline is implemented as a LangGraph StateGraph with five nodes, one conditional branch, and one bounded retry loop:

- link_entities → [if zero seed entities found] → fallback_linker → expand_subgraph
- link_entities → [if seeds found] → expand_subgraph (skips the fallback)
- expand_subgraph → [if a relation filter produced zero edges, and not yet retried] → retry expand_subgraph without the filter
- expand_subgraph → attach_documents → generate_answer → END

The shared state object (a TypedDict) carries: question, seed_entities, relation_filters, used_fallback, retried_without_filter, subgraph_nodes, subgraph_edges, graph_facts, documents, answer, and retrieved_entity_ids (the last of which feeds the recall calculation in Section 7).

5.3 The Mock Knowledge Graph

Defined in data/mock_data.py as the single source of truth for both pipelines — 33 nodes across six entity types, connected by 63 edges across 10 relation types:

Entity Type	Count	Examples
Person	8	Nina Castellanos (Compliance Lead), Arjun Mehta (Sr. ML Engineer), Grace Adeyemi (VP Engineering)
Project	5	Signal Detection Platform, SOC2 / Part 11 Audit Q1 2026, Usage-Based Pricing Redesign

Skill	5	ML Engineering, Regulatory Affairs (GxP/Part 11), Security & Access Control
Tool	6	PyTorch, Snowflake, AWS Audit Manager, Okta, LangChain/LangGraph, Tableau
Document	5	Q1 2026 SOC2 Audit Final Report, Pricing Strategy Memo, Information Security Policy v4
Decision	4	Adopt 3-tier pricing model, Standardize on PyTorch, Mandate MFA for production access
Total	33	63 directed edges across WORKED_ON, USED_TOOL, HAS_SKILL, REQUIRES_SKILL, AUTHORED, REFERENCES, DECISION_FOR, APPROVED, PROPOSED, OWNS

This satisfies the handout's 20+ node requirement with headroom, and the 10-relation-type design gives the GraphRAG pipeline meaningful relation-filtered traversals to demonstrate (vs. a graph with only one or two edge types, where filtering would add little).

5.4 Technology Stack and Rationale

Component	Choice	Why
Graph library	NetworkX (MultiDiGraph)	Pure Python, zero infra, supports multiple parallel edges between the same two nodes (useful since e.g. a person can both WORK_ON and OWN the same project) and directed traversal with relation-type metadata on each edge.
Vector store	FAISS (IndexFlatIP, CPU)	Standard, fast, in-memory baseline; flat index is appropriate at 33 chunks (no need for approximate search structures).
Embeddings	sentence-transformers all-MiniLM-L6-v2 (local, 384-dim)	Runs on CPU with no API key and no per-call cost — appropriate for a small, fixed, 33-chunk corpus; avoids adding a second paid API dependency.
Orchestration	LangChain + LangGraph	Required by the Track 2 brief; LangGraph's StateGraph cleanly models the conditional fallback-linker branch and the bounded relation-filter retry as explicit graph edges rather than nested if/else logic.
LLM (generation + linking)	Groq — llama-3.1-8b-instant (default), configurable to llama-3.3-70b-versatile for the entity linker	Fast, free-tier-friendly inference for both the GRAPH_ANSWER / VECTOR_ANSWER generation calls and the ENTITY_EXTRACTOR structured-output call. See note below on the handout's original Nebius requirement.
UI	Streamlit	Matches the chosen build track's recommended demo tooling; fast to iterate,

		good for a side-by-side comparison layout with an embedded graph plot.

A note on the original Nebius Token Factory requirement

The Week 2 handout specifies that both build tracks must route at least one model call through Nebius Token Factory, for cross-cohort comparison. This build instead uses Groq for chat/structured output and local sentence-transformers for embeddings, because a Nebius API key was not available during development. The architecture isolates all model calls inside llm.py, so swapping back to Nebius (or adding it for one call, e.g. embeddings) is a small, contained change rather than a redesign. This substitution should be flagged to the instructor/cohort lead.

6. Streamlit UI Walkthrough

The Streamlit app (ui.py) is the primary demo surface: it runs both pipelines for a chosen question and displays the answers, recall scores, and the underlying knowledge graph side by side. Every interactive element is explained below.

6.1 Page Header

- Title — “GraphRAG vs Vector RAG”.
- Caption — a one-paragraph explanation that both pipelines answer the same question about Veritask Sciences; GraphRAG traverses the knowledge graph while Vector RAG retrieves the top-k most semantically similar chunks, so the contrast shows when relationships matter.

6.2 Sidebar Controls

Element	Type	What it does / why it's there
Pick a benchmark question	Dropdown (selectbox)	Lets the user choose one of the 10 pre-defined benchmark questions (Section 7). Pre-defining questions (rather than free text) keeps the demo grounded in questions with known gold-entity sets, so recall can be computed live.
“Why this question”	Expander	Collapsible panel showing (a) the reasoning behind the selected question — which graph relationships it is designed to exercise (e.g. “Decision -DECISION_FOR-> Project, then Person -APPROVED-> Decision”) — and (b) the full gold entity set used for scoring. This is the main teaching aid: it shows the audience why a question is hard for vector search.
Run both pipelines	Primary button	Triggers both pipelines for the selected question: builds/loads the FAISS index (cached after first run), compiles the LangGraph app (cached), then calls answer_with_vector_rag and answer_with_graph_rag. Results are stored in Streamlit session state so they persist when the user changes the highlight toggle without re-running the pipelines.
Graph legend	Color-coded badges	Static legend mapping each of the six entity types to a color used in the graph visualization: Person (blue), Project (green), Skill (purple), Tool (orange), Document (yellow), Decision (red).
Graph node/edge count	Caption text	Shows “Graph has 33 nodes and 63 edges” — a quick sanity check that the full knowledge graph (not a subset) is loaded.
Highlight on the graph	Radio buttons	Controls which entities are drawn larger / highlighted in the graph plot: “Vector RAG retrieved” (the 8 nodes whose chunks were top-k matches), “GraphRAG traversed” (every node in the expanded subgraph), “Union” (both sets combined — the default, useful for seeing overlap and divergence at a glance), or “None” (plain graph, no highlighting).

6.3 Knowledge Graph Visualization

A NetworkX spring-layout plot (rendered via Matplotlib) of the full 33-node graph. Node color encodes entity type (per the legend); node size encodes whether that entity was retrieved or traversed by either pipeline for the current question, per the Highlight toggle. Edges are drawn with light, semi-transparent arrows so the overall structure is visible without overwhelming the labels. This view lets the audience see, visually, how far GraphRAG had to traverse from the seed entities to answer a multi-hop question, and how much of the graph Vector RAG's top-8 chunks actually cover.

6.4 Results Panel (appears after “Run both pipelines”)

Two columns, one per pipeline, each containing:

Element	Type	What it does / why it's there
Recall vs gold set	Metric (st.metric)	Shows the percentage of the question's gold entity names that appear in this pipeline's retrieved or traversed entity set — computed live, not pre-baked. This is the headline number for the “when each approach wins” analysis.
Missed: ...	Caption (conditional)	If recall is below 100%, lists exactly which gold entities were not retrieved — makes failure modes concrete rather than just a percentage.
Answer	Markdown + write	The LLM-generated answer text for that pipeline (the VECTOR_ANSWER or GRAPH_ANSWER prompt output).
Retrieved chunks (Vector RAG only)	Expander	Lists the 8 chunks FAISS returned as the top-k matches, each shown as “Entity name: chunk text” — lets the audience see exactly what context the LLM had to work with.
Traversed subgraph facts (GraphRAG only)	Expander	Shows the raw GRAPH FACTS text block passed to the LLM: an ENTITIES list (with seed nodes marked) and a RELATIONSHIPS list of source -RELATION-> target triples from the expanded subgraph.
Seeds / Fallback linker used	Caption	Shows which entity IDs were used as traversal seeds, and whether the deterministic fallback linker had to be used (i.e. the primary LLM linker returned zero seeds). This makes the conditional branch in the LangGraph state machine visible to the audience.

6.5 Empty State

Before the first run, the results panel is replaced with an info banner: “Pick a question and click Run both pipelines to compare results.” so the app is never shown with broken or empty placeholders.

7. Benchmark Questions and Evaluation Approach

Per the handout's deliverable, we defined 10 original benchmark questions against the Veritask Sciences graph, each with a hand-curated “gold” set of entity names that a complete, correct answer should mention. Six questions (Q1, Q2, Q5, Q6, Q8, Q10) are designed to require multi-hop relationship traversal — the kind of question GraphRAG should win on. Four questions (Q3, Q4, Q7, Q9) mix an attribute lookup with one relationship hop, where both pipelines may perform reasonably and the comparison is more nuanced.

#	Question	Gold Entities
Q1	Who worked on the SOC2 audit and what tools did they use?	Nina Castellanos, Devon Park, AWS Audit Manager, Okta
Q2	What decisions were made about the pricing model and who approved them?	Adopt 3-tier usage-based pricing model, Samira El-Amin, Priya Subramaniam
Q3	Which project is the Signal Detection Platform's architecture spec related to, and who authored it?	Signal Detection Platform (Pharmacovigilance), Arjun Mehta, Leo Fischer
Q4	What skills does Oliver Tan have and which project uses those skills?	Oliver Tan, Regulatory Affairs (GxP/Part 11), Regulatory Submission Assistant
Q5	Who approved the decision to mandate MFA for production systems?	Grace Adeyemi, Mandate MFA for all production system access, Devon Park
Q6	What tools were standardized for ML training pipelines and who proposed that decision?	PyTorch, Standardize on PyTorch for ML training pipelines, Arjun Mehta
Q7	Which documents reference the Data Quality Remediation project, and who worked on it?	Adverse Event Data Quality Runbook, Data Quality Remediation for Adverse Event Feeds, Leo Fischer, Priya Subramaniam
Q8	Who is the VP Engineering and which decisions did they approve?	Grace Adeyemi, Mandate MFA for all production system access, Standardize on PyTorch for ML training pipelines
Q9	What is the status of the Regulatory Submission Assistant project and who owns it?	Regulatory Submission Assistant, Priya Subramaniam, Oliver Tan
Q10	Which security tools are used across projects and who is the security engineer?	Devon Park, Okta, AWS Audit Manager, Information Security Policy v4

7.1 Scoring Method

For each question and each pipeline, recall is computed as the number of gold entity names present in the pipeline's retrieved (or traversed) entity set, divided by the total number of gold entity names for that question.

For Vector RAG, the retrieved entity set is the set of node names corresponding to the top-8 FAISS matches. For GraphRAG, it is the set of all node names present in the final expanded subgraph (retrieved_entity_ids in the LangGraph state). Both compare.py (CLI) and ui.py (Streamlit) compute this

live against the same questions.py gold sets, so the numbers shown in the demo are reproducible from the same definitions documented here.

7.2 Validation of the Entity-Linking Safety Net

Before involving any LLM calls, we validated the deterministic fallback linker (regex over node names plus a topic-shortcut map for terms like “audit”, “SOC2”, “MFA”, “pytorch”, and “VP Engineering”) combined with 2-hop subgraph expansion, with no LLM in the loop. Result: 100% recall across all 10 questions using the fallback path alone. This means the LangGraph pipeline has a robust floor even if the primary LLM-based entity linker (llama-3.1-8b-instant on Groq) returns an empty or malformed JSON response — a realistic risk with a small instant model.

8. Prompts, Iterations, and Learnings (Vibe-Coding Log)

8.1 Prompt Library

The three prompt templates used by the system (defined across llm.py, graph_builder.py, graph_rag.py, and vector_rag.py):

VECTOR_ANSWER (vector_rag.py, used for the Vector RAG baseline's generation step)

Instructs the model to answer using only the retrieved chunks, to name every relevant entity by full name, and to explicitly say so if the context is insufficient — implementing the “I don't know” path required by the framework.

GRAPH_ANSWER (graph_rag.py, used for GraphRAG's generation step)

Tells the model to treat the GRAPH FACTS block (entities and relationship triples from the expanded subgraph) as the primary source of truth, and the SUPPORTING DOCUMENTS block as secondary prose detail only — mirroring the reference kit's design principle that structured graph facts should out-rank free text when they conflict or overlap.

ENTITY_EXTRACTOR (graph_builder.py, used by the primary LLM entity linker)

Asks the model to extract (a) entity names mentioned or implied in the question and (b) which of the 10 relation types the question is asking about, returned as JSON matching a fixed schema. The extracted names are then fuzzy-matched (via difflib) against the 33 node labels to produce seed node IDs.

8.2 Build Iterations

Summary of the major iterations during development, in order:

1. Drafted the RAG framework (Section 3) and chose the fictional company (Veritask Sciences) and domain before writing any code, per the handout's guidance to scope the problem first.
2. Designed the 33-node, 63-edge mock dataset (data/mock_data.py) with 10 relation types, explicitly cross-referencing entities across People, Projects, Tools, Documents, and Decisions so that multi-hop questions would have real answers in the graph.
3. Authored 10 original benchmark questions and gold entity sets (questions.py), split between multi-hop relationship questions and single-hop attribute questions, to support the “when each approach wins” analysis required by the handout.
4. Implemented graph_builder.py: graph construction, the LLM entity linker, the deterministic fallback linker (with a topic-shortcut map), 2-hop subgraph expansion with relation filtering, and subgraph-to-text serialization.
5. Implemented vector_rag.py (FAISS baseline) and graph_rag.py (LangGraph state machine), adapting the reference kit's five-stage GraphRAG flow (build graph, link entities, expand subgraph, attach documents, generate answer) to our schema and relation types.
6. Built compare.py (CLI recall report) and ui.py (Streamlit side-by-side app with graph visualization), modeled on the reference kit's CLI / Streamlit split but redesigned for our entity types and highlight modes.
7. The initial model-provider plan was Nebius Token Factory (per the handout's requirement). When a Nebius key was unavailable, llm.py was refactored to route chat and structured-output calls through Groq and embeddings through a local sentence-transformers model — isolated to one file so the rest of the pipeline was unaffected.
8. Validated the deterministic fallback linker end-to-end against all 10 gold sets without any LLM calls, confirming 100% recall as a safety net (Section 7.2) before wiring up the LLM-based primary linker.

9. Fixed a LangGraph state-passing bug: an internal NetworkX subgraph object was being passed between nodes under a non-declared state key (_subgraph). LangGraph's TypedDict-based StateGraph silently drops undeclared keys, causing a KeyError in the attach_documents node. Fix: the subgraph is now reconstructed from the declared subgraph_nodes list (via g.subgraph(node_ids)) inside attach_documents, rather than passed by reference.

8.3 Learnings and Observations

- Designing the dataset and the benchmark questions together, before writing any retrieval code, made it much easier to guarantee that gold answers actually existed in the graph — several early draft questions were reworded once we noticed the graph didn't have a direct edge to support them.
- A small, deterministic fallback linker (regex plus a topic map) is disproportionately valuable: it achieved 100% recall on its own across all 10 questions, meaning the system degrades gracefully even when the LLM-based linker (especially a small “instant” model) returns malformed or empty JSON.
- LangGraph's TypedDict state schema enforces a useful discipline — every piece of information that needs to flow between nodes must be explicitly declared — but objects that can't or shouldn't be serialized (like a live NetworkX subgraph) need to be either declared explicitly or reconstructed downstream from declared primitives, as we did.
- Treating graph facts as primary truth and documents as supporting detail in the GRAPH_ANSWER prompt, rather than concatenating everything with equal weight, reduced the chance of the model over-indexing on document prose at the expense of the structured relationships the graph was specifically built to surface.
- Swapping the model provider (Nebius to Groq plus local embeddings) was straightforward because all model calls were isolated in a single llm.py module from the start — a useful pattern for any RAG project where the provider may change.
- The relation-filter retry in expand_subgraph (drop the filter if it returns zero edges) mattered in practice: several questions' relation-type guesses from the LLM linker were close but not exact (e.g. expecting DECISION_FOR when the more useful edge was APPROVED), and the retry recovered a useful subgraph anyway.

9. Repository Structure and How to Run

9.1 Files

File	Purpose
data/mock_data.py	Mock org knowledge base: 33 nodes + 63 edges (single source of truth for both pipelines).
graph_builder.py	Builds the NetworkX graph; LLM + fallback entity linkers; 2-hop subgraph expansion; subgraph-to-text and document-attachment helpers.
graph_rag.py	LangGraph state machine implementing the GraphRAG pipeline.
vector_rag.py	FAISS-based vector RAG baseline pipeline.
llm.py	Chat (Groq) and embedding (local sentence-transformers) wrappers; the single place model providers are configured.
questions.py	10 benchmark questions with gold entity sets and rationale (the “why” field).
compare.py	CLI: runs both pipelines over all 10 questions, prints answers and recall, and an overall summary.
ui.py	Streamlit comparison app (Section 6).
PROJECT_PLAN.md	Original planning document: framework, architecture, dataset design, and benchmark question rationale.
README.md	Setup and run instructions.
requirements.txt, .env.example, .gitignore	Dependencies and environment configuration (no real secrets committed).

9.2 Setup and Run

From the project root:

- python3 -m venv .venv && source .venv/bin/activate
- pip install -r requirements.txt
- cp .env.example .env, then add your GROQ_API_KEY (get one from console.groq.com/keys)
- CLI comparison: python compare.py
- Streamlit demo: streamlit run ui.py

The first run downloads the all-MiniLM-L6-v2 embedding model (about 90MB) for local use; subsequent runs use the cached model.

9.3 Deliverables Checklist (Mapped to This Document)

Handout Requirement	Where Covered
Knowledge graph with 20+ nodes	Section 5.3 (33 nodes)
GraphRAG vs vector RAG comparison on 10 queries with recall	Section 7,

	compare.py
Analysis of when each approach wins	Sections 4 and 7.2
Project documentation: overview, datasets, prompts, iterations, learnings	Sections 2, 5, 8
Streamlit demo	Section 6, ui.py
Designated inference provider for at least one model call	Section 5.4 (Groq substitution noted)
Video demo (5 min or less) and GitHub repo	To be recorded and published by the student